

Aidge: A comprehensive and traceable library for deploying standalone embedded deep neural networks

Cyril Moineau, Maxence Naud, Iryna De Albuquerque Silva, Axel Farrugia, Olivier Bichler

▶ **To cite this version:**

Cyril Moineau, Maxence Naud, Iryna De Albuquerque Silva, Axel Farrugia, Olivier Bichler. Aidge: A comprehensive and traceable library for deploying standalone embedded deep neural networks. ERTS 2026 - 13th European Congress of Embedded Real Time Systems, Feb 2026, Toulouse, France. ⟨10.82331/ERTS.2026.42⟩. ⟨hal-05513843⟩

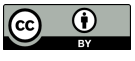
HAL Id: hal-05513843

https://hal.science/hal-05513843v1

Submitted on 16 Feb 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Aidge: A comprehensive and traceable library for deploying standalone embedded deep neural networks

Cyril Moineau, Maxence Naud, Iryna de Albuquerque Silva, Axel Farrugia and Olivier Bichler

Univ. Paris-Saclay, CEA LIST, F-91120, Palaiseau, France
firstname.lastname@cea.fr

ABSTRACT

Embedding deep neural networks (DNNs) in resource-constrained systems is gaining increasing interest across various domains. However, deployment remains hindered by opaque, vendor-specific tools that limit transparency, flexibility, and reproducibility issues. In this paper, we introduce Aidge, an open-source framework for the design and deployment of DNNs in constrained environments. Aidge is built around four core principles: being community-driven and dependency-free, offering a user-friendly hierarchical graph intermediate representation, enabling full traceability, and ensuring two-way interoperability with major deployment tools. These principles guide a modular architecture that allows users to analyze, optimize, and deploy models with a high degree of control and portability. By reducing reliance on black-box toolchains and supporting transparent, verifiable workflows, Aidge offers a practical and robust foundation for embedded AI development, particularly in applications requiring high assurance, such as safety-critical systems.

1. INTRODUCTION

Deep learning has demonstrated remarkable effectiveness across a wide range of domains, including computer vision, natural language processing, autonomous systems, and medical diagnostics. However, Deep Neural Network (DNN) applications often come with high memory and computational demands, posing challenges for resource-constrained environments. Recent advances in model design and optimization, such as iViT [30] and MobileNet [41], made it increasingly feasible to deploy DNNs on embedded platforms, thereby enabling real-time, on-device inference across a growing number of applications.

However, reliance on vendor specific tools remains a major enduring limitation, especially in the context of safety-critical applications. Implementation details are often undisclosed to users, effectively rendering these tools as "black boxes". Furthermore, these Software Development Kits (SDK) often come with their own compression method which can also be obscured or directly wired to the toolchain with no possibilities to retrieve the resulting DNN. Such variability can result in significant differences in application performance for the same DNN deployed across multiple targets. Eventually, in a target exploration context, the diversity of these SDKs may constitute a time-consuming learning overhead. In this paper, we present the Aidge framework [34], an open-source deep learning platform specialized in the design of deep neural networks intended to operate in systems constrained by power consumption or dissipation, latency, form factor (dimensions, size, etc.), and/or cost criteria. Aidge addresses these limitations by providing a transparent, compiler-like deployment flow that generates static, model-specialized C++/C code rather than relying on a generic interpreter. This enables fine-grained control over memory planning, data layouts and computational kernel selection. Unlike vendor-specific SDKs, all internal passes, operators, and optimizations are transparent and modifiable, which facilitates hardware exploration, deterministic behavior and certification-oriented use cases.

Cyril Moineau et al.. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

2. RELATED WORK

There exist many machine learning frameworks dedicated to the deployment of DNN models. These frameworks exhibit diverse configurations and functionalities, but mainly focus on optimizing the DNN model to improve the average execution time or memory usage on the target hardware. The majority of deployment engines follow a common compiler structure: (i) the front-end converts a trained model into an IR and (ii) the back-end then generates a machine-code of the model optimized for a given target. Some of these frameworks employ an interpreter approach, relying on dynamic interpretation or just-in-time compilation to achieve efficient inference execution. Other employ a code generation approach, either exposing the model implementation in a source code, written in widely-used languages like C or C++, which is later compiled to suit the application requirements, or directly generating a binary file. Hereafter, we present different approaches and analyze their compatibility with Aidge's funding principles.

Compiler-based deployment frameworks *OpenVINO* [24] targets Intel hardware (CPUs, GPUs, VPUs, FPGAs) and uses a workflow that converts trained models (e.g. from TensorFlow, PyTorch, ONNX) into a proprietary IR for optimization, compilation, and execution via its open-source runtime. *Vitis AI* [27] is designed for AMD platforms, particularly FPGAs, and similarly transforms models into an IR, applying quantization and pruning before compiling them into microcode for FPGA-based acceleration; however, its compiler and runtime are proprietary. *TensorRT* [33], optimized for NVIDIA GPUs, performs model conversion, quantization, and fusion before generating an optimized runtime engine that runs on top of CUDA and cuBLAS; like Vitis AI, it is not open-source.

LiteRT [22], previously named TensorFlow Lite [5], is TensorFlow's inference framework for running machine learning models on embedded systems, such as ARM CPUs, Android and iOS devices. The engine takes as input a special compressed format (FlatBuffer [44]) and is an interpreter-based engine. Although the execution code is static, the interpreter handles the model structure dynamically, defining which operators to execute and where to retrieve the model parameters. The authors argue that this approach, as opposed to native code generation, improves deployment portability, as the same FlatBuffer model can be executed across heterogeneous platforms without per-model recompilation. *Apache TVM* [10] is a compiler framework based on LLVM [15] that supports models from various ML frameworks and enables hardware-agnostic deployment. It includes AutoTVM for automated optimization and extends to microcontrollers via microTVM [13]. *microTVM* adapts the TVM workflow to run without an OS, using a C runtime and one of two executors: a memory-efficient ahead-of-time (AOT) executor, without access to source code, or a Graph executor, requiring JSON parsing and dynamic memory allocation. *Qualcomm AI Engine Direct* [37] provides a workflow for deploying pretrained neural network models optimized for Qualcomm hardware (e.g. CPU, GPU, NPU). The toolchain converts models from third-party frameworks (e.g. ONNX, TFLite, PyTorch) into an intermediate runtime format consisting of a C++ graph representation and a binary weight file. It supports quantization to multiple integer precisions depending on hardware capabilities. The converted model can be compiled either into a dynamic library for inference or, in a static manner, through an offline compilation step that generates a context binary cache and simplifies the initialization of the model. *ExecuTorch* [20] is a deployment framework designed for executing PyTorch models on edge and embedded devices through an AOT compilation pipeline. Models are lowered to a compact serialized artifact that encodes the optimized computation graph, quantized tensors, and memory planning. Inference is then performed by a minimal, modular runtime implemented in C++, which can be embedded into the user's application. The framework is open source, but only the runtime and kernel implementations are accessible. The exported model artifact itself remains a compact binary format rather than generated source code. None of these solutions offer transparent or traceable modifications between the input model and the final executable code, which are common requirements for deploying AI-based software in safety-critical systems.

Code-generating deployment frameworks *MATLAB Coder with Deep Learning Toolbox* [45] implements the DNN model in a C or C++ code, either generic or optimized for *Intel* and *ARM* processors. This toolchain takes as input ONNX and TensorFlow models and requires a first conversion to a MATLAB model object. However, as a license-based proprietary tool, MATLAB does not publicly disclose its IR or optimization details. *Ansys Neural Networks Package* [6] is an engine to generate C code for DNN models targeting safety-critical applications. The toolchain includes a Keras importer that generates a SCADE [12] model, which is then translated into C code that can be deployed independently of any specific runtime. However, we were not able to access their solution and verify if transformations preserve the model semantics. *VkCoreVX* [14] is CoreAVI's proprietary toolchain for deploying neural networks in safety-critical systems using GPU acceleration via Vulkan SC. It supports models trained in TensorFlow, PyTorch, which are converted to NNEF and then compiled into Vulkan SC format for deployment. While it claims to target certification needs (e.g. DO-178C, ISO 26262), the details of its optimizations are undocumented, limiting transparency and independent verification. Moreover, its reliance on proprietary, GPU-specific infrastructure makes it less suited to our objectives of openness, portability, and reproducibility. *X-CUBE-AI* [43] is STMicroelectronics' toolchain that converts pretrained neural networks into C code optimized for STM32 microcontrollers. It supports models from established frameworks such as Keras, LiteRT, and ONNX (including Pytorch and MATLAB exports), and provides optional acceleration through the Neural-ART hardware NPU. The toolchain supports various quantization pipelines and efficient memory mapping to enable larger models. However, as a vendor-specific solution tightly coupled to STM32 hardware, X-Cube-AI focuses on model deployment rather than transformation transparency. Its internal representation and optimization passes are not publicly documented, and its portability is limited to the STM32 ecosystem. *Plumerai's* Inference Engine [35] is a proprietary runtime designed for improved execution of quantized neural networks on resource-constrained devices, leveraging SIMD and AI accelerators. Built on top of TensorFlow Lite for Microcontrollers (TFLM), it replaces the generic interpreter with model-specific and precompiled C++ code generated by its Inference Engine Generator. Its design centers on quantized data formats (INT8 and INT16) for most layers' operations, while non-quantized INT32 and FLOAT32 types can be used for support operations. Nonetheless, it is a closed-source commercial product, and its internal generator and runtime are not publicly accessible.

3. FUNDING PRINCIPLES

The Aidge framework is focused on broad accessibility and application-centric design that translate into the following key principles:

Community-Driven and Dependence-Free Deep learning frameworks often suffer from excessive reliance on third-party dependencies. These dependencies, often maintained by third-party vendors or single contributors, can lead to long-term maintenance and compatibility issues. Aidge mitigates these risks by grouping its extensions into the project's code repository, ensuring they are well-maintained and accessible. This approach does not eliminate the need for hardware-specific kernels when targeting particular devices, but it ensures that the core workflow and generic operators remain self-contained, maintained within the repository. Furthermore, being hosted by the Eclipse Foundation [16], Aidge avoids reliance on a single entity, fostering a sustainable ecosystem. The repository encompasses a wide range of extensions, from neural network import to optimization, analysis, and target-specific export, making Aidge versatile for both research and industrial applications.

User-Oriented Hierarchical Graph IR In opposition to more developer-oriented Intermediate Representation (IR) such as MLIR [29], Aidge employs a user-centric, hierarchical graph

IR that prioritizes ease of manipulation and extensibility. Its object hierarchy is simple, with a clear, distinct role for each class of the IR that can be customized. Moreover, the overall structure of the framework was made modular to separate most key functionalities into their own optional module with minimal interdependencies. New modules can easily be added with templates provided for a quick start.

Transparency for Traceability Frameworks, such as TensorRT [33], apply opaque transformations to the DNN during their compilation processes making it difficult for the user to assess or debug the result. Aidge, on the other hand, prioritizes transparency for its graph-level operations, and total access to the graph at any step of the workflow. Every transformation is associated with an explicit update of the graph topology and dataflow. This approach guarantees that the operations applied to the graph are fully transparent, enabling users to trace and verify each modification. Such transparency is essential for fostering trust, reproducibility, and compliance with rigorous standards, particularly in safety-critical applications where the integrity of model transformations must be meticulously documented and audited. For instance, in the aeronautics domain, standards such as AIR6988 [40] and DO-178C/ED-12C [19] explicitly mandate these properties for software development.

Two-way Interoperability Aidge accessibility is a top priority. Its C++ codebase is fully exposed to Python with a Pythonic interface, enabling users to leverage the framework's capabilities in a high-level, user-friendly environment. Furthermore, many frameworks only support one-way conversion either to or from the state-of-the-art ONNX exchange format [8]. Aidge, however supports both import and export of operators in a two-way interoperability. This allows a great compatibility with the broader ecosystem of deep learning tools. Since the graph is fully available at any point in the transformation pipeline, it can be retrieved and converted to ONNX. Moreover, the generated code is standalone, with no dependency to the Aidge platform nor to a runtime. This property increases its compatibility with users existing applications. Besides, Aidge's bidirectional ONNX interoperability also enables design-space exploration. Since every transformation applied to the graph is transparent and accessible, users can export intermediate versions of the model to ONNX for external analysis, profiling, or verification, and then reimport refined variants back into the platform.

4. Aidge's software architecture

4.1. Context

The Aidge framework [34] has been developed as a successor to N2D2 [9]. These two platforms provide a complete toolchain from graph modelization, training, compression, optimization and code generation to target a specific hardware device. With N2D2 being to our knowledge the first public deep learning framework to integrate transcoding and simulation of spiking DNNs, as well as dedicated hardware code generation. Figure 1 summarizes the Aidge deployment workflow.

4.2. Aidge Intermediate Representation

The Aidge framework represents a DNN model as a directed computational graph, where vertices correspond to computational operations and edges define the flow of data between these operations. This graph is explicitly constructed, either through the Aidge API or by importing from formats such as ONNX. In contrast to imperative execution models (e.g. PyTorch eager mode), where the computational graph must be reconstructed by tracing program execution, Aidge follows a static-graph paradigm: the graph is defined directly rather than inferred. Consequently, each operator appears in the graph exactly as specified, without implicit fusion or rewriting unless deliberately invoked.

4.2.1. Multi-Paradigm Abstraction Model

The graph vertices are split into three layers of abstraction: the *Node*, the *Operator* and the *Implementation*.

Nodes carry the topological information of the graph. Each node maintains references to its outgoing and incoming edges, which connect it to their children and parents nodes, respectively. These connections represent the data flowing between nodes. The term data is deliberately broad as it can be specialized into different types, such as tensors, sparse tensors or spikes.

Operators are data structures that characterize the operation associated with a *Node*. They hold both specific attributes to the operation and the effective input and output data. Since it is free from any inference computation, the *Operator's* structure is agnostic to how it is coded. We reserve the term of *parameters* for *learnable parameters*, e.g. weights and biases of Convolutions. These are defined by a special kind of *Operator* called *Producers*. The introduction of *Producers* allows to explicitly represent the data in the graph. Therefore, we use the term *attributes* to refer to the values that configure the operation (e.g. the stride of a Convolution). An *Operator* can be shared by multiple *Nodes*, in order to avoid memory duplication. Aidge defines a set of built-in *Operators* which can be extended by users. Each one identified by a type, which is a unique string per kind of *0perator* (e.g. `"Conv2D"`).

Implementations hold the computational kernel, i.e. how the operation is actually coded and define how data is consumed during the computation process. This separation allows a graph to be defined and manipulated before providing an actual implementation. The *Operator* selects the *Implementation* to use at graph-construction time via a registration system. This selection depends on the expected data type (the precision and representation of data, e.g. INT8, FLOAT32...) and data format (the memory layout e.g. NCHW (Batch, Channel, Height, Width), NHWC (Batch, Height, Width, Channel), ...) of both inputs and outputs. Other factors can also impact the *Implementation* selection, such as specific attribute values (e.g. a kernel size of 3x3 for a Convolution). The choice of implementation is resolved within the Aidge framework prior to code generation, ensuring that no dynamic dispatch occurs in the final deployed binary. After code generation, the resulting C/C++ code contains only static calls to the selected kernels, with no runtime implementation lookup on the target device. The registration mechanism can also be extended with predicate functions to express more complex selection rules when needed.

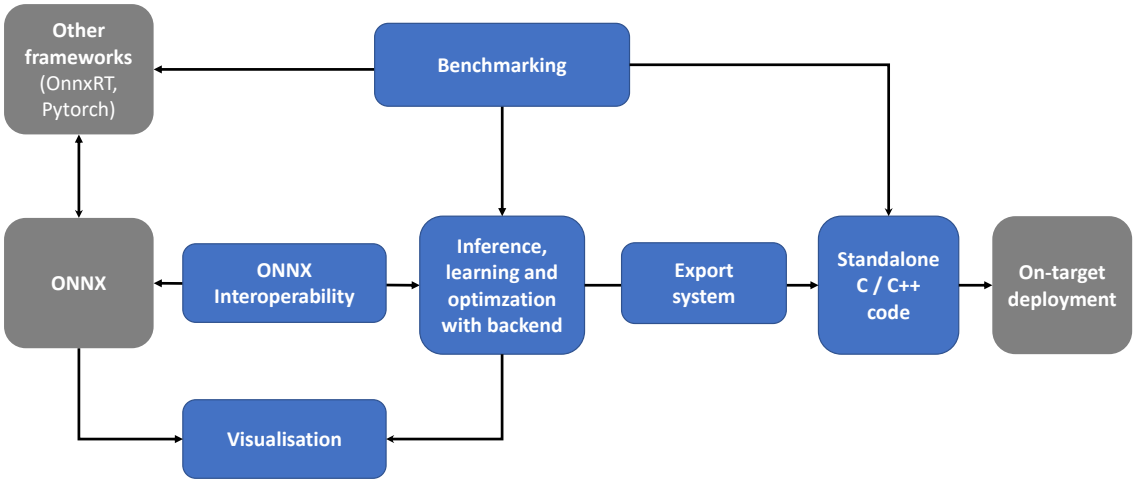


Figure 1. Overview of the Aidge deployment workflow. Blue tiles represent Aidge functionalities and grey tiles represent elements outside of Aidge.

4.2.2. Hierarchical Model

Some operations can be defined as a composition of other operations. To avoid the complexity of handling multiple operators that can be redundant or arbitrarily fused together (e.g. the Padding and the Convolution), we introduce the concept of *MetaOperator*. This *Operator* embeds a graph, called *micrograph* and allows the user to define a type. A *MetaOperator* can be executed using a dedicated *Implementation* or by combining the *Implementations* of the *Operators* inside its micrograph. It is possible to easily switch between viewing a *MetaOperator* as a single element or as its detailed micrograph with just one command. This change of granularity enables the usage of high-level hardware-specific features, while also allowing finer-grained transformations, such as fusing *Operators* that would otherwise be obfuscated.

4.2.3. Incomplete Graph Model

While Aidge defines an extensive set of operators, it also introduces a frame for custom operations called *GenericOperator*. A *GenericOperator* can be defined on the fly by specifying a custom type, the number of inputs and outputs, and attributes. This flexibility allows users to load and manipulate any graph without errors, even when working with operators not defined in the built-in set. To complete their workflow, a user may supply a *GenericOperator* with a custom implementation written in Python. A *GenericOperator* behaves like any other base *Operator* in Aidge, and fully integrates with all Aidge functionalities.

4.2.4. Searchable Graph Model

In order to easily manipulate the graph, Aidge introduces a graph description formalism with a Regular Expression (ReGex)-like syntax for matching graph patterns. This set of rules is named *GraphRegEx*. An expression written with this syntax, coupled with graph matching algorithms, allows to easily retrieve every subgraph that matches the set of patterns described by the expression. *GraphRegEx* uses the types defined in the *Operators*. The syntax allows to match any graph topology: sequential, parallel and recurrent. Furthermore, quantifiers and wildcard symbols are available to match complex graphs. *GraphRegEx* can also match *Operators* based on their attributes.

Here is an example expression that matches a GeLU operation:

```
Div#1->Erf->Add#1-*>Mul->Mul#2;
Div#1<1~Producer;
Add#1<*~Producer;
Mul#2<*~Producer;
```

Listing 1. GraphRegEx for GeLU.

4.3. Aidge Toolbox

Leveraging this robust graph representation, Aidge provides an extensive toolbox that enables users to manipulate, analyze, schedule, and export the computation graph.

4.3.1. Graph Transformations

Aidge offers convenient one-liner methods for manipulating the computation graph. Some examples include:

Insertion: adds and connects a *Node* in the graph;
Replace: substitutes a set of *Nodes* with another one, preserving connections if possible;
Remove: disconnects a *Node* from the graph, reconnecting when possible its parents and children;
Factorization: replaces a subgraph with a *MetaOperator*, which encapsulates the subgraph;
Expansion: replaces a *MetaOperator* with the subgraph it encapsulates.

In addition, the framework provides a set of predefined transformation *recipes*, such as constant-folding, operator fusion, and identity node elimination, as well as a *cleaner* utility that automatically applies selected or default recipes. These mechanisms enable fully automated simplification of the computation graph when desired. We emphasize that the explicit graph-manipulation API is made available to preserve transparency and to allow advanced users to control transformations when exploring alternative graph structures or validating safety-critical pipelines, but it is not a prerequisite for deployment. At each

step, the user can visualize the state of the graph using different approaches, such as writing a Mermaid [31] file, generating a JSON file, or using the AI Edge Model Explorer tool [23].

4.3.2. Graph Scheduling & Memory Mapping

This section details how Aidge approaches scheduling, defines a memory mapping and enables a static analysis of deep learning graphs. Aidge introduces a consumer-producer (C-P) model for *operator implementations*, similar to transaction-level modeling for electronic design in order to schedule the graph.

The scheduling of a graph is defined here as an ordered list of operators to call to compute the forward pass. The C-P model is tied to an *operator implementation*, which specifies the amount of data consumed and produced at each execution step, i.e. at each forward pass.

For example, if an implementation consumes the whole tensor then the resulting scheduling will be equivalent to a topological sorting of the graph. However, if we use another implementation which now only consumes part of the input (e.g. an horizontal band), the C-P model would then allow the scheduler to be a pipelined dataflow.

Moreover, the C-P model can be specified from precise amounts of data (number of elements) or arbitrary data quantities (*tokens*). Tokens are used when input and output sizes are unknown without an execution. As a result, even when tensor sizes are unknown, the graph can still be reliably scheduled.

Decoupling the C-P model from the data execution allows to schedule the graph execution without providing the actual *Operator*'s implementation. Aidge's base *Scheduler* uses this C-P model to statically schedule a graph before execution. The scheduling is then accessible by the user, either by directly accessing the call stack, or after execution. Indeed, it is possible to export the scheduling with a Gantt chart in Mermaid format containing the execution time of each *Operator*.

This scheduling is also used for the code generation, paired with a memory mapping. Aidge generates a static memory layout. It does so by mapping the outputs and inputs of each *Operator* into a single global memory space. This memory mapping is based on the scheduling done beforehand. While the user can perform their own custom memory mapping, Aidge proposes two built-in strategies. The first one is a naive approach which places each output sequentially in memory without reusing any space, resulting in a simple, contiguous layout. The second approach, which is the default memory mapping, supports memory reuse (reallocating previously used memory space) and memory wrapping (looping memory addresses to fit tensor shapes). Internally, Aidge's memory management can handle any data format, while being optimized for NHWC data layouts.

Figure 2 shows an example of memory mapping for a ResNet8 [4] model on a single memory unit. The x-axis, labeled "Time", represents the logical steps of the scheduled graph, whereas the y-axis measures memory usage. Each block represents a memory allocation for an operator's outputs. The memory peak of the NN is reported. This example is particularly interesting as it features a graph with branches; consequently, the first block is retained across two logical steps.

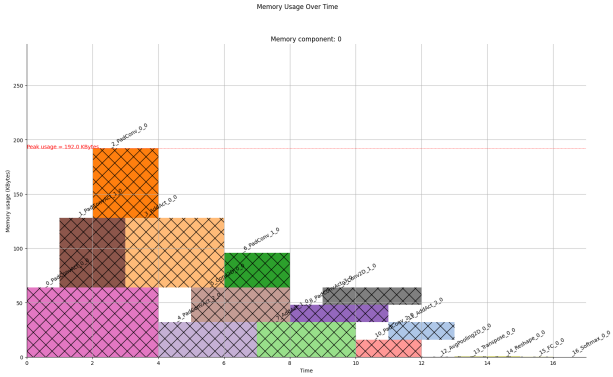


Figure 2. Example of memory mapping for a ResNet8 for a heterogeneous memory.

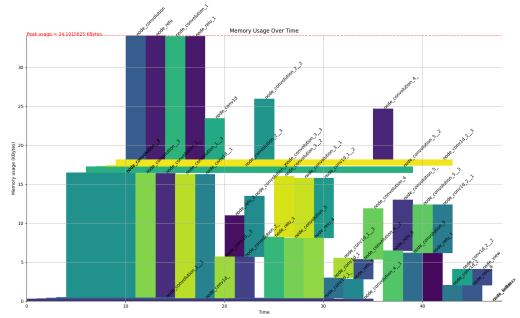
4.3.3. Compression methods

Aidge implements several compression methods, ensuring transparency during the process. This means that the user is able to follow, at each step, the transformations which are applied to the graph.

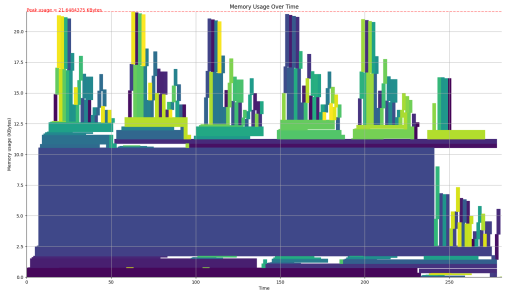
The optimization methods in Aidge can be categorized as follows:

Quantization: reducing the precision of weights and activations to 8bits integer and below. Aidge implements a classic Post Training Quantization method (no retraining of the model) as well as Quantization Aware Training (QAT) method such as SAT [25] and LSQ [18]. One of the particularity of Aidge is that the user can customize the quantization scheme layer-wise by choosing the precision of the weights, activations and accumulation and enabling mix precision. The scaling operation can also be changed (floating point vs single shift scaling, with or without rounding) to fit specific usecases or hardware constraints. Finally, Aidge allows to run the graph with True Quantization (effective INT8 representation and not FLOAT32 with INT8 dynamic) which allows to have a golden model with bit accurate results.
Tiling: splitting the input of an operator in order to improve data locality or parallel execution (not to be confused with compiler tiling which is loop wise). In the Fig 3 example, tiling the operators allows to reduce the memory peak usage by about 38%.
Tensor decomposition: factorizing large tensors into lower-rank components to reduce parameter count and computation cost. Aidge implements a distribution aware tensor decomposition method [26].
Pruning: removing redundant or insignificant parameters or operations to produce a sparser and more efficient model. Similar to automatic quantizer insertion for QAT, Aidge also provides pruner operators that can be automatically inserted into the graph to perform pruning of weights and/or activations.

This list of compression methods is only a snapshot of what is currently available and we are actively working on introducing new methods. As an example, Aidge also implements innovative state of the art methods such as Non Linearity Removal [39] which allows to remove non linearity in the graph in order to fuse linear operations together.



(a) Memory mapping before tiling



(b) Memory mapping after tiling

Figure 3. Comparison of a neural network before and after Tiling. Source: [17].

4.3.4. Template Engine for Code Generation

To support hardware deployment, Aidge includes a flexible code generation framework. Aidge generates open-source code which can then be compiled for a hardware target. In order to generate code, Aidge uses the Jinja template engine. Each *Node* is linked to a function that generates code using the same selection system used for *Implementations*. The generated code describes the computational functions tailored to a specific *Operator* configuration. This code is referred to as a *kernel*. A file named *forward.cpp* is also generated, which invokes each kernel function in the order defined by the static scheduler, storing intermediate results according to the memory mapping strategy. A built-in structure is provided for C/C++ projects. Owing to the utilization of Jinja templates, the code generation process can be easily extended to other programming languages beyond C/C++ with minimal effort.

4.4. Software Design Overview

As mentioned, one distinctive aspect of Aidge is that it is based on the principle of *modularity*, i.e. there is a *core* module that can be extended by plugins, allowing the straightforward addition of new features. Figure 4 illustrates the overall software design of Aidge and how its components interact.

The *Core* module is developed entirely in C++ (14) with bindings to Python ($>$ 3.10) and includes a set of functions to: create a computational graph representing a DNN, manipulate it (e.g. by deleting, replacing or adding a *Node*) and query it to match a specific sequence of *Operators*.

Plugins, which can be developed in C++ with a Python binding or in pure Python, are categorized as:

Backend: registers *Implementations* to the `aidge_core` module;

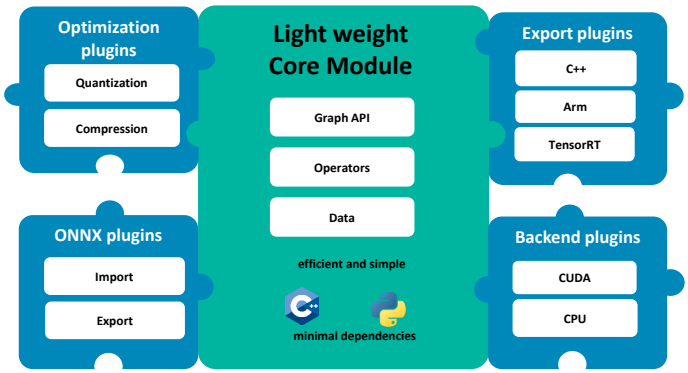


Figure 4. Overview of Aidge's modular software design, with a central core and multiple plugins that extend frontends, optimizers, and backends.

Export: exposes methods to adapt the graph to a target hardware, and produces source code for the corresponding graph;
Transformation: implements *recipes* that manipulate the graph;
Optimization: implements algorithms that reduce the computation and memory costs of the DNN (e.g. quantization, tensor decomposition, pruning ...);
Interoperability: enables conversion to and from Aidge's IR and other frameworks such as ONNX, PyTorch, or Google's Model Explorer. These plugins support model import, export, and cross-framework compatibility.

Extending Aidge with these plugins is then straightforward, as a simple Python import is enough to register new *Implementations* to the *Core* module.

5. Comparative Evaluation

Aidge is shipped with its own cross-library benchmarking tool. It enables to assess the time performances and correctness of either a complete DNN model or individual operators. The benchmarking tools are available in two variants: a command-line interface (`aidge_benchmark`) and a Python API, both of which enable fast and consistent comparisons across multiple deployment frameworks using a shared test configuration description.

Benchmarking an operator requires a user-provided test configuration file that specifies the operator to evaluate together with the input properties and attribute variations to test. A configuration file is structured in three parts: (i) meta-information describing the operator type, *opset* version, and general testing options; (ii) a base configuration defining default input shapes and attributes; and (iii) a collection of test configurations, each overriding selected parts of the base configuration to explore different parameter settings (e.g. feature-map sizes or kernel shapes). Pre-made configuration files are also provided for the most commonly used operators.

In order to benchmark an entire DNN model with Aidge, two inputs are required: a model file in a supported format (ONNX) and its inputs as NumPy arrays. Then, a set of functions registered for each desired framework backend computes the output of the model and the inference time. These computation functions come with a corresponding set of evaluation and visualization functions.

Table 1. Summary of experimental setup for benchmarking Aidge.

	Target HW	Precision	Criteria assessed	Toolchains
B1	x86	FLOAT32	Inference time Numerical correctness	`backend_cpu` ONNX Runtime PyTorch

B2				`export_cpp`
				`export_xnnpack`
				ONNX Runtime

B3	Raspberry PI 4	FLOAT32	Inference time
			Numerical correctness
				`export_cpp`
				`export_xnnpack`
				ONNX Runtime

B4	STM32H743ZI	FLOAT32	Inference time Numerical correctness	`export_arm_cortexm` X-CUBE-AI TensorFlow Lite Micro
B5		INT8		`export_arm_cortexm` X-CUBE-AI TensorFlow Lite Micro

5.1. Evaluation criteria

Inference times are compared by executing each model 50 times per input and computing all pairwise ratios between the inference times of two frameworks. The median of these ratios is then used as the comparison metric. This approach is more computationally expensive but provides increased robustness to outliers and yields a more reliable estimate of the central tendency.

Model *numerical correctness* is assessed by computing absolute and relative output differences relative to a reference framework. These differences are then validated against specific user-defined tolerances for every input.

5.2. Experimental setup

To evaluate the effectiveness of Aidge across diverse application domains, we conducted experiments on four representative models selected from the MLCommons Tiny Inference Benchmark [32] suite: (i) binary image classification (visual wake-word) on Visual Wake Words Dataset [11] using MobileNetv1 [3], (ii) keyword spotting on Speech Commands [46] using DS-CNN [2] (iii) image classification on CIFAR-10 [28] using ResNet8 [4], and (iv) anomaly detection on the ADMOS Toy Car dataset [36] using a lightweight Dense network [1]. We report inference time and numerical correctness under multiple deployment settings. Table 1 summarizes the experimental setup for the comparative evaluation.

In this section, the *backend* term is used to describe a set of generic computational kernels that are used by a given framework to run inferences on a host device. In contrast, an *export* refers to generated code tailored for a specific operator configuration, for example, by leveraging the C++ templating mechanism.

This section evaluates several Aidge backends and export modules. The `backend_cpu` provides a generic C++ implementation designed for execution within the Aidge framework on x86 architectures. Although Aidge also includes a CUDA backend, our experiments focus exclusively on CPU-based backends to ensure a more relevant comparison within the context of hardware export. We compare `backend_cpu` with PyTorch, a well known DNN Framework as well as ONNX Runtime on a computer equipped with an x86-64 processor. ONNX Runtime utilizes multiple optimized backends, such as XNNPACK [21], for efficient model execution.

For the benchmarking of standalone deployments, we evaluate two specific export modules. The `export_cpp` module generates generic standalone C++14 code without utilizing hardware intrinsics or making assumptions regarding the target architecture. In contrast, `export_xnnpack` provides an optimized alternative by leveraging the XNNPACK kernel library. These exports are evaluated on an x86-64 processor as well as a Raspberry Pi 4 Model B [38], equipped with a quad-core ARM Cortex-A72.

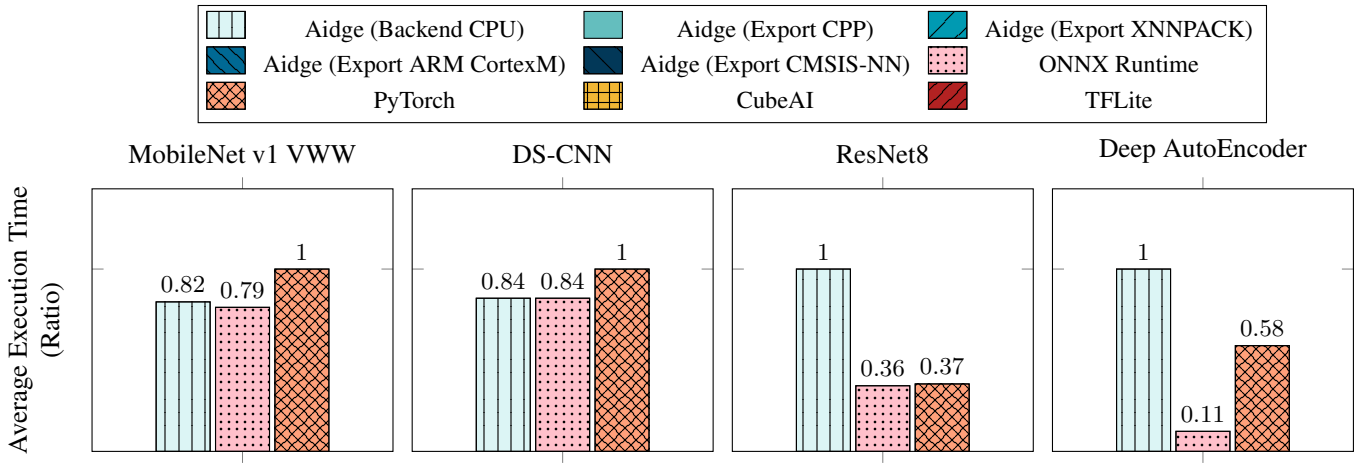
Finally, the `export_arm_cortexm` module is tailored for code generation targeting STM32 platforms. For quantized models, this export module integrates the optimized CMSIS-NN [7] library to maximize performance on Arm Cortex-M processors. For this benchmark, we use the STM32H743ZI [42] target to evaluate the models on both FLOAT32 and INT8 precisions. The `export_arm_cortexm` generated export is compared with the STMicroelectronics X-CUBE-AI and TensorFlow Lite Micro (TFLM) toolchains, both using the CMSIS-NN library for quantized models.

5.3. Results

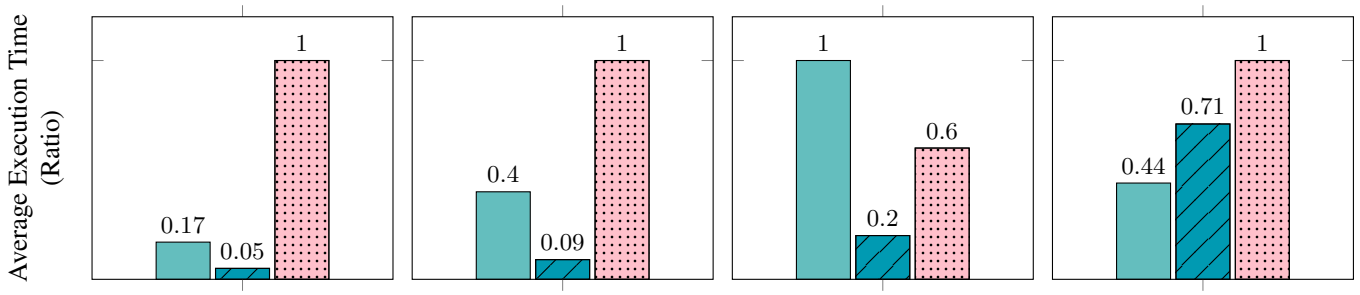
Inference time

The first configuration, i.e. benchmark *B1* in Table 1, evaluates FLOAT32 inference time on a x86-64 processor. This setup compares the Aidge `backend_cpu` against established desktop inference engines such as ONNX Runtime and PyTorch. As we can see in Figure 5a, depending on the model, Aidge `backend_cpu` has comparable performances with other frameworks. While it is not the main feature of Aidge, the `backend_cpu` is necessary for checking the model correctness and to optimize it before export. The primary goal of this benchmark is to showcase that Aidge is not lagging behind the state-of-the-art.

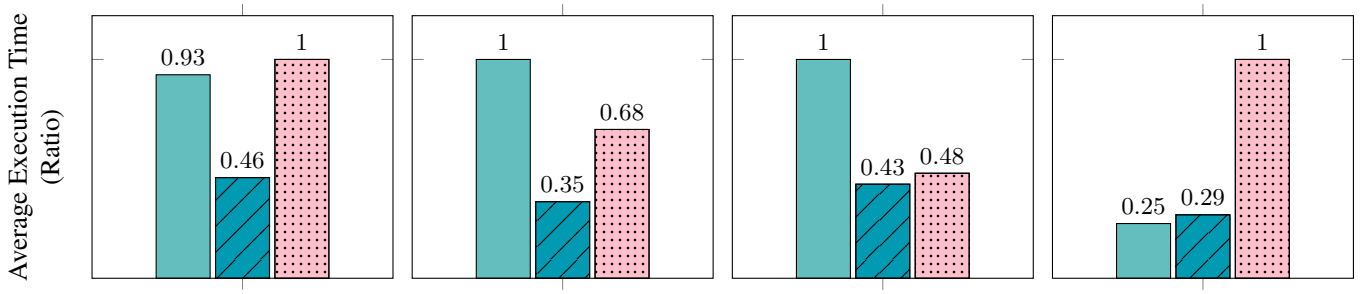
The second configuration, i.e. benchmark *B2*, follows the same setup as the first one (FLOAT32 on x86) but evaluates



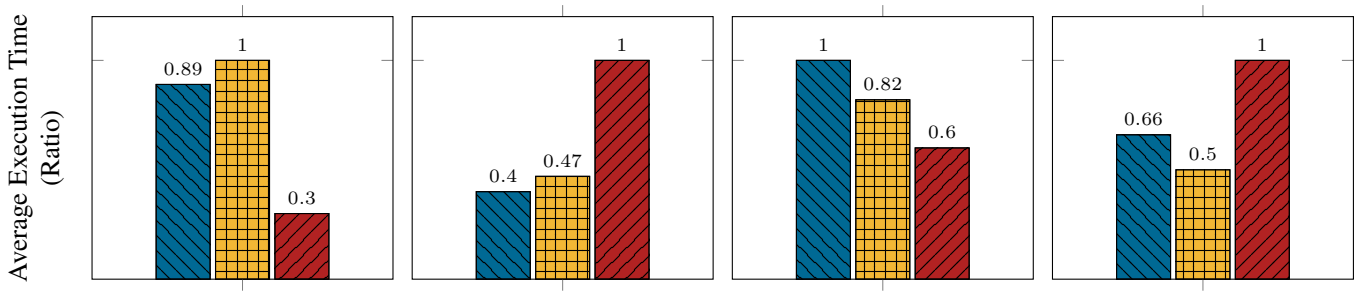
(a) *Benchmark 1*: Average execution time ratios for runtimes on x86-64 processor (500 warm-ups + 2,000 iterations).



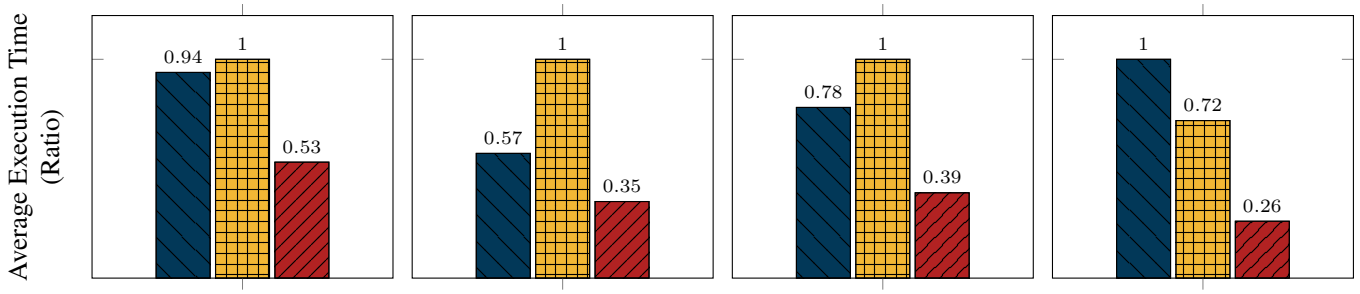
(b) *Benchmark 2*: Average execution time ratios for exports on x86-64 processor (500 warm-ups + 2,000 iterations).



(c) *Benchmark 3*: Average execution time ratio for exports on Raspberry 4 Model B (100 warm-ups + 1,000 iterations).



(d) *Benchmark 4*: Average execution time ratio for exports FLOAT32 on STM32H743ZI (2 warm-ups + 50 iterations).



(e) *Benchmark 5*: Average execution time ratio for exports INT8 on STM32H743ZI. (2 warm-ups + 50 iterations).

Aidge's `export_cpp` and `export_xnnpack` export modules against ONNX Runtime as the reference baseline. Figure 5b shows the differences in performance between exported code and a runtime. ONNX Runtime which was outperforming other runtimes is now the one being outperformed.

This benchmark should be put into context with the Figure 5c which shows that on a specific hardware such ARM A72, which possesses an ARM Neon accelerator, a generic C++ code with a simple compiler (in this instance GCC) fails to close the gap with platform-specific solutions such as ONNX Runtime, which uses libraries such as XNNPACK. XNNPACK in particular possesses acceleration specific to ARM Neon architecture. This explains why ONNX Runtime is comparable or better than `export_cpp` whereas on a x86-64 the trend was inverted.

This highlights the distinct advantage of Aidge's export design: the ability to generate code that maps Aidge operators directly to optimized kernels. This strategy is critical for closing the performance gap relative to specialized libraries, thereby ensuring competitive execution speeds without compromising the essential framework features of Aidge. In this example both `export_cpp` and `export_xnn` were made out of the same graph, the difference is the computation kernel used to compute the operations.

The benchmark *B4* results, Figure 5d, shows a relative similarity between Aidge's and CubeAI's execution times. However there is no clear pattern emerging from the TFLM results as, depending on the model, it can be twice as slow or twice as fast. The INT8 experiments (benchmark *B5*), in Figure 5e, show that the Aidge toolchain achieve better results than the CubeAI one on most of the evaluated models. However the TFLM toolchain appears to be the most effective for each model.

Numerical correctness

The numerical validation confirmed that the model outputs remained highly consistent across all evaluated frameworks. For Benchmarks *B1*, *B2*, and *B3*, ONNX Runtime served as the reference implementation, while CubeAI was used as the baseline for Benchmarks *B4* and *B5*. In all test cases, both absolute and relative differences were contained within a strict margin of 10^{-5} compared to the respective reference, ensuring that the model's precision was preserved across all platforms and export methods.

6. Conclusions

In this work, we introduced Aidge, a comprehensive deployment framework tailored for standalone inference of deep neural networks on embedded platforms. Aidge is designed around four core principles: being community-driven and dependence-free, offering a user-oriented hierarchical graph intermediate representation, ensuring transparency for full traceability, and supporting two-way interoperability with major machine learning deployment frameworks. These guiding values shaped both the architecture and tooling of the framework, enabling users to efficiently manipulate, analyze, optimize and deploy DNNs across a range of embedded environments. By aligning with these principles, Aidge not only addresses practical deployment needs but also promotes robustness, verifiability and maintainability, making it well-suited for use in safety-critical environments. Beyond its technical contributions, Aidge is fully open-source and hosted by the Eclipse Foundation, ensuring long-term governance, transparency, and community sustainability. The project provides a growing set of tutorials and examples that guide users through the core components of the platform, from graph construction and transformation to backend selection and code generation, thereby promoting accessibility for users with varying levels of expertise.

Acknowledgments

This work was supported by the DEEPGREEN project under grant agreement ANR-23-DEGR-0001 and the NEUROKIT2E European Union's Horizon Europe research and innovation program under grant agreement No 101112268. The authors gratefully acknowledge the contributions of all project partners.

References

[1] Mlcommons tiny benchmark – dense model (anomaly detection, admos toycar). https://github.com/mlcommons/tiny/blob/master/benchmark/training/anomaly_detection/trained_models/ad01_fp32.tflite, . Accessed: 2025-10-30.

[2] Mlcommons tiny benchmark – pretrained ds-cnn (keyword spotting). https://github.com/mlcommons/tiny/blob/master/benchmark/training/keyword_spotting/trained_models/kws_ref_model_float32.tflite, . Accessed: 2025-10-30.

[3] Mlcommons tiny benchmark – pretrained mobilenet (visual wake words). https://github.com/mlcommons/tiny/blob/master/benchmark/training/visual_wake_words/trained_models/vww_96_float.tflite, . Accessed: 2025-10-30.

[4] Mlcommons tiny benchmark – pretrained resnet (cifar-10). https://github.com/mlcommons/tiny/blob/master/benchmark/training/image_classification/trained_models/pretrainedResnet.tflite, . Accessed: 2025-10-30.

[5] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, et al. TensorFlow: Large-scale machine learning on heterogeneous systems. https://www.tensorflow.org/, 2015. Software available from tensorflow.org.

[6] Ansys. Neural Networks Package V1. https://catalog.ansys.com/product/620eb1f7475fc9e881614971/neural-networks-pa, 2022.

[7] ARM-software. CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs. https://github.com/ARM-software/CMSIS-NN, 2017–2025. Accessed: 2025-10-30.

[8] Junjie Bai, Fang Lu, Ke Zhang, et al. onnx. https://github.com/onnx/onnx, 2025. GitHub Repository, version v1.18.0, accessed 2025.

[9] CEA-LIST. N2D2. https://github.com/CEA-LIST/N2D2, 2023. GitHub Repository, version v1.3.0, accessed 2025.

[10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: end-to-end optimization stack for deep learning. *CoRR*, abs/1802.04799, 2018.

[11] Aakanksha Chowdhery, Pete Warden, Jonathon Shlens, Andrew Howard, and Rocky Rhodes. Visual wake words dataset. *arXiv preprint arXiv:1906.05721*, 2019.

[12] Jean-Louis Colaço, Bruno Pagano, Cédric Pasteur, and Marc Pouzet. Scade 6: From a kahn semantics to a kahn implementation for multicore. In *2018 Forum on Specification Design Languages (FDL)*, pages 5–16, 2018.

[13] TVM consortium. microTVM: TVM on bare-metal. https://tvm.apache.org/docs/topic/microtvm/index.html, 2021.

[14] CoreAVI. VkCoreVX SC and SafeAI. https://coreavi.com/product_category/vkcorevx-sc-and-safeai-capabilities/, 2020.

[15] LLVM Developers. The LLVM Compiler Infrastructure. https://llvm.org, 2002.

[16] Eclipse Foundation. Eclipse foundation. https://www.eclipse.org/, n.d. Accessed: 2025-06-26.

[17] Eclipse Fundation. Aidge documentation, 2023. URL https://eclipse.dev/aidge/. Eclipse Aidge documentation.

[18] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization, 2020. URL https://arxiv.org/abs/1902.08153.

[19] EUROCAE/RTCA, Inc. ED-12C DO-178C - Software Considerations in Airborne Systems and Equipment Certification, 2011.

[20] ExecuTorch Developers. Executorch. https://github.com/pytorch/executorch, 2023. Accessed: 2025-10-30.

[21] Google. XNNPACK: High-Efficiency Neural Network Inference Operators for CPU. https://github.com/google/XNNPACK. Accessed: 2025-10-30.

[22] Google. LiteRT. https://ai.google.dev/edge/litert, n.d. Accessed: 2025-10-30.

[23] google-ai edge. model-explorer. https://github.com/google-ai-edge/model-explorer, 2025. GitHub Repository, version model-explorer-visualizer-npm-v0.1.1, accessed 2025.

[24] Intel. Open Vino documentation. https://docs.openvino.ai/latest/index.html, 2018.

[25] Qing Jin, Linjie Yang, and Zhenyu Liao. Towards efficient training for neural network quantization, 2019. URL https://arxiv.org/abs/1912.10207.

[26] Alper Kalle, Theo Rudkiewicz, Mohamed-Oumar Ouerfelli, and Mohamed Tamaazousti. Distribution-aware tensor decomposition for compression of convolutional neural networks, 2025. URL https://arxiv.org/abs/2511.04494.

[27] Vinod Kathail. Xilinx Vitis Unified Software Platform. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, FPGA '20, page 173–174, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450370998. doi: 10.1145/3373087.3375887. URL https://doi.org/10.1145/3373087.3375887.

[28] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[29] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 2–14, 2021. doi: 10.1109/CGO51591.2021.9370308.

[30] Zhikai Li and Qingyi Gu. I-ViT: Integer-only Quantization for Efficient Vision Transformer Inference, August 2023.

[31] Mermaid Contributors. Mermaid - Diagramming and charting tool. https://github.com/mermaid-js/mermaid, 2024. Accessed: 2025-06-26.

[32] ML Commons. MLPerf Inference: Tiny. https://github.com/pytorch/executorch, 2025. Accessed: 2025-10-30.

[33] NVIDIA. TensorRT documentation. https://developer.nvidia.com/tensorrt, 2021.

[34] Maxence N. Olivier B., Cyril M. Eclipse aidge. https://gitlab.eclipse.org/eclipse/aidge, 2023.

[35] Plumerai. Plumerai inference engine. https://docs.plumerai.com/2.3/products/inference_engine/, 2021. Accessed: 2025-10-30.

[36] Hemant Purohit, Ryuki Tanabe, Nobutaka Inoue, Keisuke Imoto, Takashi Ogawa, and Yoichi Kawaguchi. Mimii dataset: Sound dataset for malfunctioning industrial machine investigation and inspection. In *DCASE Workshop*, 2019.

[37] Qualcomm Technologies, Inc. Qualcomm ai engine direct sdk documentation. https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-10/general_introduction.html, 2025. Accessed: 2025-10-30.

[38] Raspberry Pi Foundation. Raspberry Pi 4 Model B Specifications, 2023. URL https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/.

[39] Baptiste Rossigneux, Vincent Lorrain, Inna Kucher, and Emmanuel Casseau. Importance resides in activations: Fast input-based nonlinearity pruning. In *International Conference on Neural Information Processing*, pages 32–44. Springer, 2024.

[40] SAE. AIR6988 - Artificial Intelligence in Aeronautical Systems: Statement of Concerns, 2021.

[41] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks, March 2019.

[42] STMicroelectronics. Stm32h743zi — microcontrollers and microprocessors, 2025. URL https://www.st.com/en/microcontrollers-microprocessors/stm32h743zi.html.

[43] STMicroelectronics. X-Cube-AI: AI Expansion Package for STM32. https://www.st.com/en/embedded-software/x-cube-ai.html, 2025. Accessed: 2025-10-30.

[44] TensorFlow. TensorFlow Lite FlatBuffer Model. https://www.tensorflow.org/lite/api_docs/cc/class/tflite/flat-buffermodel, 2020.

[45] Inc. The MathWorks. MATLAB Deep Learning Toolbox, 2023. URL https://fr.mathworks.com/help/coder/deep-learning-with-matlab-coder.html.

[46] Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition. *arXiv:1804.03209*, 2018.